Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

SAVED: Sensitive Operation-Aware Vulnerability Detection model Enhanced with Distance-Weighted Sparse Attention

Kaiyang Cai
Jilin University
Jinghao Hu
Jilin University
Jingyao Li
Jilin University
Zhanshan Li
lizs@jlu.edu.com

Jilin University

Research Article

Keywords: Computer Security, Software Security, Code Vulnerability, Software Vulnerability, Vulnerability Detection, Vulnerability Localization

Posted Date: July 10th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-6887893/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License.
Read Full License

Additional Declarations: No competing interests reported.

SAVED: Sensitive Operation-Aware Vulnerability Detection model Enhanced with Distance-Weighted Sparse Attention

Kaiyang Cai[1], Jinghao Hu[2], Jingyao Li[1*], Zhanshan Li[1*]

[1*]College of Computer Science and Technology, Jilin University, Changchun, 130012, Jilin, China.
[2]College of Software, Jilin University, Jilin University, Changchun, 130012, Jilin, China.

*Corresponding author(s). E-mail(s): jingyao@jlu.edu.cn; lizs@jlu.edu.com;
Contributing authors: tsaiky1125@gmail.com; hjh22@mails.jlu.edu.cn;

Abstract

Software security has received increasing attention due to the rapid growth in software vulnerabilities and the significant damage they cause. Although various deep learning-based vulnerability detection models have been developed, these models typically operate at the function or code slice level, introducing numerous irrelevant statements into predictions. Moreover, current techniques lack reliable mechanisms to explain the detected vulnerabilities. To address these limitations, we propose **SAVED** (Sensitive operation-Aware Vulnerability detection model Enhanced with Distance-weighted sparse attention) inspired by a common cognition: *vulnerabilities always arise from insufficient validation before or after sensitive operations*. SAVED is an end-to-end Graph Neural Network (GNN) model that detects vulnerabilities in source code and provides interpretable explanations. Specially, SAVED automatically constructs a subgraph for each sensitive operation using our noval Distance-Weighted Sparse Attention (DWSA) mechanism, which mitigates the noise from irrelevant statements. It then computes a vulnerability score for each node by integrating semantic features, operation-specific information, and the generated sensitive subgraphs, thereby providing direct insight into the model's decision-making process and facilitating the localization of vulnerability-related statements. Extensive experiments demonstrate that SAVED outperforms current state-of-the-art methods and provides reliable explanations that enable accurate root-cause localization.

Keywords: Computer Security, Software Security, Code Vulnerability, Software Vulnerability, Vulnerability Detection, Vulnerability Localization

1 Introduction

The incidence of software vulnerabilities has significantly increased in recent years. According to the National Vulnerability Database (NVD), the number of reported vulnerabilities rises from 6,487 in 2015 to 39,990 in 2024, and shows a persistent upward trend [1]. Traditional vulnerability detection methods [1–3] rely heavily on human expertise and provide an incomplete coverage of potential vulnerabilities. In contrast, deep learning-based models [4–10], particularly Graph Neural Network (GNN) models, emerging as promising methods for vulnerability detection. These models eliminate reliance on expert-crafted patterns and significantly reduce both false positive and false negative rates. Typically, they represent source codes as graphs, such as Code Property Graph [11] (CPG) or Program Dependency Graph [12] (PDG), or as multiple code slices based on points of interest (POI), and employ GNNs, including Gated Graph Neural Network [13] (GGNN) and Graph Convolutional Network [14] (GCN), to extract semantic and structural features. Despite their effectiveness, these models still exhibit several limitations.

Coarse-grained node initialization masks critical operations. Existing methods commonly initialize graph nodes by pre-trained word embeddings (e.g., Word2Vec [15], Node2Vec [16]). Although this supports general semantic understanding, it obscures the fine-grained operational details crucial for vulnerability identification [17].

Global graph pooling blurs vulnerability-specific features. Existing methods employ global graph pooling over all nodes to derive graph embeddings for the final prediction. However, vulnerabilities typically involve a small subset of nodes. Figure 1 compares total source code lines to vulnerable statement lines (Outer), and illustrates the proportion of vulnerable statements within the source code (Inner), demonstrating that vulnerable statements constitutes a small fraction of the overall code. Global graph pooling introduces substantial noise, encouraging models to memorize high-level patterns rather than learn genuine vulnerability characteristics.

Post-hoc explanations are disjointed from the detection process. To aid security analysts, some approaches integrate auxiliary explanation models (e.g., GNNExplainer [18]) or compute node importance scores through parameter analysis [9, 19] after training to interpret the detection process. However, these methods typically yield *post-hoc* explanations that are disjointed from the detection, often resulting in interpretations that are difficult to align with the underlying detection mechanism [20, 21].

In this paper, we propose SAVED (Sensitive operation-Aware Vulnerability detection model Enhanced with Distance-weighted sparse attention), an end-to-end

[1] https://nvd.nist.gov/

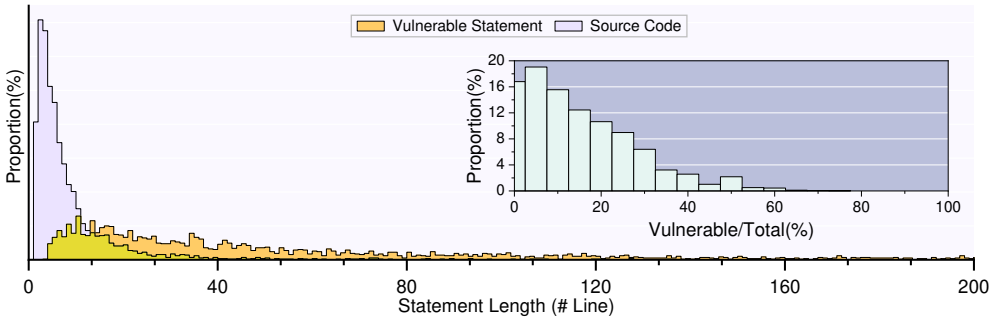


Fig. 1: Statistic of distribution of total source code lines versus vulnerabile statement lines(Outer) and the proportion of vulnerable statements within the source code.

GNN vulnerability detection model that prioritizes sensitive operations in code and simultaneously detects and locates vulnerabilities.

Operation-semantic composite node feature initialization. Effective vulnerability detection requires accounting for both implementation details and context. First, Different operations imply different constraint requirements. For example, assignment operations require extra caution with regard to operand types, whereas array declarations define variable bounds that subsequent operations may exceed. Second, vulnerabilities are highly context-dependent. For instance, an integer overflow may be benign during hash calculation but can pose a serious threat during memory allocation. To capture both aspects, we initialize node features by integrating semantic embeddings with operation-specific attributes.

Attention to sensitive operations. Language models (LMs) of code[22–26] exhibit impressive performance in tasks such as code generation and comprehension for their powerful semantic understanding capabilities. Nevertheless, relying solely on semantic understanding limits their effectiveness in vulnerability detection [27]. Figure 2 (a) and (b) illustrate the code before and after the patch for CVE-2019-14816 [2]. Although both versions implement the same functionality, the patched version incorporates additional validation before the `memcpy` operation. Consequently, LMs that ignore structural and logical dependencies may generate predictions primarily based on code functionality, overlooking crucial structural differences. In contrast, our findings indicate that Large Language Models (LLMs) with advanced reasoning abilities demonstrate strong performance in vulnerability detection, as they can assess whether sufficient constraints surround sensitive operations during inference. Figure 2 (c) presents responses from Qwen2.5-coder-7B and DeepSeek-R1-7B when prompted to "Review the code for potential security vulnerabilities." To perform this task, both models first pay particular attention to the sensitive `memcpy` call, examine its parameters, and identify insufficient validation of the destination buffer `ie->ie_buffer` before delivering their assessments. Inspired by the reasoning process of LLMs for vulnerability detection, we develop a sensitive operation-aware vulnerability detection model. Specially, we design the novel Distance-Weighted Sparse Attention (DWSA) module

[2]https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-14816

which guides GNN models to automatically verify weather adequate constraints are imposed for sensitive operations.

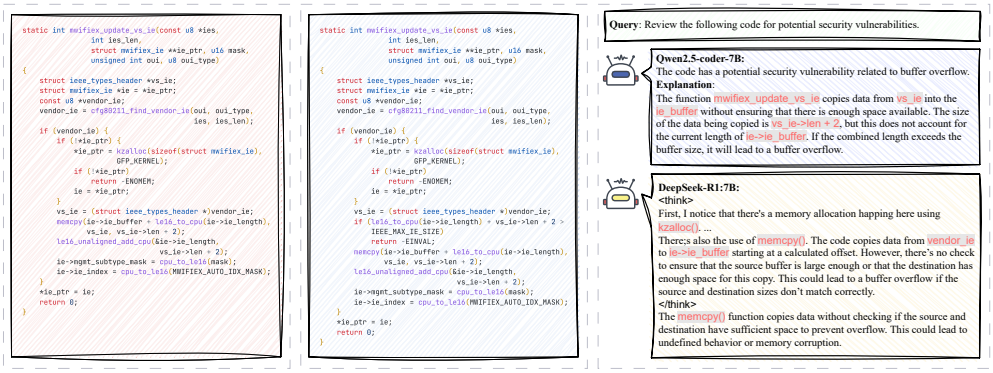


Fig. 2: **(a)** The vulnerable version of CVE-2019-14816; **(b)** The patched version of CVE-2019-14816; **(c)** The reasoning and vulnerability detection predictions generated by Qwen2.5-coder-7B and DeepSeek-R1-7B models.

End-to-end interpretation for prediction. Unlike previous methods that produce post-hot explanations, SAVED derives explanation directly from the detection process. When making prediction, SAVED explicitly integrates semantic features, operation-specific details, and sensitive subgraphs into a unified prediction layer, providing direct insight into the model's internal reasoning. This internal reasoning can directly serve as explanation for prediction, aiding analysts in pinpointing vulnerability-related statements.

In summary, the contributions of this paper are as follows:

- We propose a novel distance-weighted sparse attention (DWSA) mechanism that guides GNN models to verify weather adequate constraints surround sensitive operations.
- We propose SAVED, an end-to-end GNN vulnerability detection model that prioritizes sensitive operations in code and provides predictions along with corresponding explanations.
- We conduct extensive experiments on three real-world datasets and compare our approach with current state-of-the-art models, demonstrating the effectiveness of SAVED's detection and interpretability.

2 Preliminaries

2.1 Basic Concepts

Code Dataset: We define a code dataset as a collection of $|m|$ samples, denoted by $\{(c_i, y_i)\}_{i=1}^{|m|}$, where $c_i \in C$ represents a code sample and $y_i \in Y$ is the corresponding label, with 1 indicating vulnerable and 0 indicating non-vulnerable.
Code Graph Representation: A code sample is represented as a graph $g(V, X, A) \in G$, where $V = \{v_1, v_2, \cdots, v_n\}$ is the set of nodes, and $X \in \mathbb{R}^{n \times d}$ is the initial node feature matrix. Each node $v_i \in V$ is encoded as a d-dimensional real vector $x_i \in \mathbb{R}^d$, with n denoting the total number of the nodes. Additionally, $A \in \mathbb{R}^{n \times n \times k}$ is the adjacent matrix describing the edge connection information, where $a_{i,j,k} = 1$ denotes an edge of type k from node v_i to node v_j.

2.2 Problem Definition

Vulnerability Detection: We define the vulnerability detection as a binary classification problem, with the objective of training a model to learn a function$f : G \to Y$, which maps a graph representation g of code sample c to its corresponding label y.
Interpretation of the Prediction: The interpretation of the prediction involves identifying the minimal set of critical nodes that contributes to the positive prediction. The objective is formulated as:

$$V' = \underset{\substack{V' \subseteq V \\ f(V|V')=y}}{\arg\min} |V'|$$

3 Methodology

This section presents the framework of the SAVED, as illustrated in Figure 3. The framework consists of four main components:

1. **Code Graph Representation** (Section 3.1): Node features are initialized by combining operation-specific one-hot features and semantic embeddings, and nodes containing sensitive operations are masked.
2. **Local Pattern Learning** (Section 3.2): The model captures local patterns using a variant of the GGNN that exploits different edge types (i.e., control flow and data dependency).
3. **Sensitive Subgraph Learning** (Section 3.3): The model employs the distance-weighted sparse attention mechanism to construct and learn sensitive subgraphs;
4. **Readout and Prediction** (Section 3.4): The model integrates semantic features, operation-specific details, and sensitive subgraphs to determine the final output.

3.1 Code Graph Representation

In the first step, we aim to transform source code to a graph representation while preserving vulnerability-related semantic and structural features. To achieve this goal,

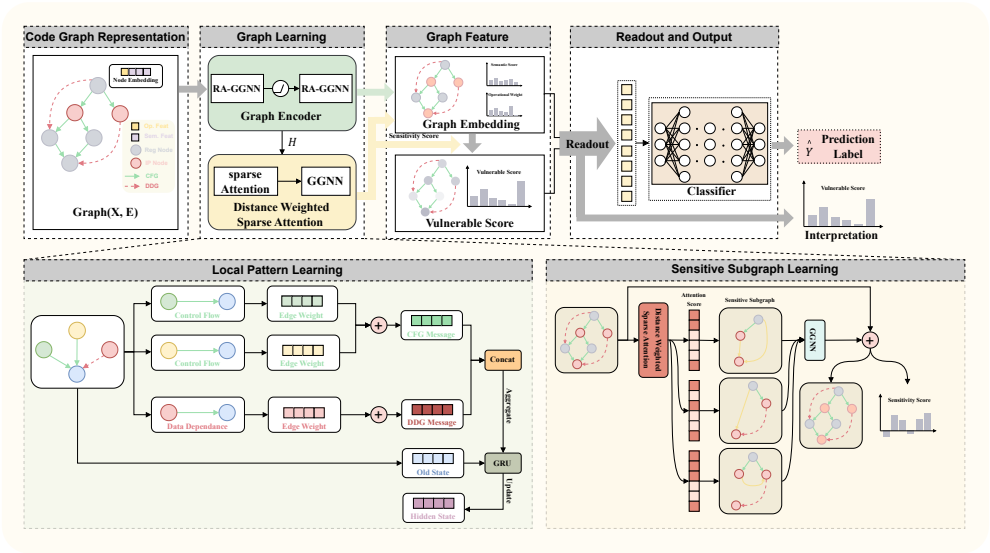


Fig. 3: The framework of our model SAVED

we first generate a Program Dependency Graph (PDG), whose nodes are connected based on control-flow and data dependency information. Notably, we did not use Code Property Graph (CPG) as prior work [5, 6] because the additional Abstract Syntax Tree (AST) heavily increases the size of the graph and exacerbates the distance between elements.

3.1.1 Operation-Semantic Composite Node Features

Upon obtaining the PDG, we need to represent each node with a proper feature embedding. Our node embeddings consist of two components: one-hot encoded features representing operation-specific details, and semantic embeddings. Specially, we extract operation-specific details for each node, such as assignment operations and array declarations. We define eleven types of one-hot features, each set to true when any corresponding operation is present. These additional one-hot features mitigate the loss of operational details during the construction of semantic embeddings and help the model focus more on specific operations. More details of the one-hot features are provided in Table 1.

Additionally, vulnerability is highly contextual-related. We treat each node as a sequence of tokens and pre-train a Word2Vec model for word embedding. Node semantic embeddings are generated by averaging all word embeddings.

As a result, the initial node features are obtained by concatenating the one-hot features with the semantic embeddings.

Table 1: Summary of the operational one-hot features

Name	Operation	Count
Sensitive API	System Call	810
Memory operation	malloc, free, $\cdots$	15
Assignment	=	1
Arithmetic assignment	+=, -=, *=, /=, %=	5
Bit assignment	&=, —=, ˆ=, >>=, <<=	5
Arithmetic operations	+, -, *, /, %	5
Logical operations	&&, ——, !	3
Bit operations	&, —, ˆ, ∼, <<, >>	6
Comparison operations	==, !=, ¿, ¡, ¿=, ¡=	6
Pointer declaration	e.g., int* p;	1
Pointer operation	e.g., p = nullptr;	1
Array declaration	e.g., int a[N];	1
Array operation	e.g., a[0] = 1;	1
Total		860

3.1.2 Sensitive Node Mask

In practice, the root cause of a vulnerability and its corresponding manifestation point constitute only a small portion of the overall code. Locating vulnerability throughout the entire code is akin to searching for a needle in a haystack. Since vulnerability semantics typically reside in specific operations (i.e., memory assesses and dangerous API calls), we treat nodes containing these operations as sensitive nodes. Following the rules proposed by [28], we define sensitive nodes as follows:

- **API Call**: Function calls to APIs from specific libraries, which are associated with vulnerabilities such as buffer overflow and use-after-free.
- **Memory Operation**: Operations on pointer variables and array-like variables, which are related to vulnerabilities such as null pointer dereference and out-of-bound access.

Unlike slice-based methods [4, 7, 9], we mask sensitive nodes not for code slicing, but rather to provide the model with a heuristic for prioritizing potentially vulnerable code. When analysing a piece of code, SAVED will first identify these sensitive nodes and verify if adequate validations are imposed to constraint the sensitive operations by constructing a sensitive graphs.

3.2 Local Pattern Learning

We employ a variant of GGNN to capture local structural information in the graphs, referred as Relation-Aware Gated Graph Neural Network (RA-GGNN). Unlike prior work [5, 6, 8] that overlooks edge types, our approach treats PDG as a heterogeneous graph with various edge types and processes message passing among nodes according to their relationship.

We now describe the RA-GGNN for heterogeneous graphs. At each iteration, the hidden states of all nodes and edges serve as inputs to the network. For a given node v, messages from neighbors are aggregated according to different edge types. Formally, the aggregated message through edge type k at iteration l is defined as

$$m_{v,k}^{(l)} = \text{Agg}(h_u^{(l-1)} | a_{u,v,k} = 1), \tag{1}$$

where $h_u^{(l-1)}$ represents the hidden state of the source node u from the previous iteration, and $m_{v,k}^{(l)}$ is the aggregated message to node v via edge type k.

The complete message to node v is then obtained by concatenating the messages from all edge types:

$$m_v^{(l)} = \text{CONCAT}_{i=1}^{k}(\mathbf{W_i} \cdot m_{v,i}^{(l)}), \tag{2}$$

with $\mathbf{W_i}$ denoting learning parameters associated with edge type i.

To update the hidden state of node v, the RA-GGNN employs a gated recurring unit (GRU) [29] that integrates the current state with the aggregated messages:

$$h_v^{(l)} = \text{GRU}(h_v^{(l-1)}, m_v^{(l)}). \tag{3}$$

3.3 Sensitive Subgraph Learning

After RA-GGNN processing, nodes in graph have captured local contextual information. However, as illustrated ahead, the model must enable fine-grained detection that assesses weather adequate constraints surround sensitive operations. To overcome these challenges, we introduce our distance-weighted sparse attention (DWSA) mechanism, which helps construct subgraphs centered on sensitive nodes. This mechanism simulates the reasoning process of LLMs that first identify sensitive nodes and then verify the constraints to sensitive operations.

3.3.1 Distance-Weighted Sparse Attention Mechanism

For a graph G_i, let $\mathbf{H} \in \mathbb{R}^{n \times d}$ denote the matrix of hidden representation after RA-GGNN processing. The mechanism first computes the attention of sensitive nodes v_p towards another node v_q, which is formulated as:

$$s_{p,q} = \log \theta_{p,q} \cdot (\frac{h_p h_q^\top}{\sqrt{d}}), \tag{4}$$

where $h_p \in \mathbb{R}^{1 \times d}$ and $h_q \in \mathbb{R}^{1 \times d}$ represent the hidden states of the sensitive node v_p and node v_q, respectively, and $\theta_{p,q}$ is the absolute distance between them. The logarithm of the distance is incorporated as weight to bias the attention mechanism toward nodes that are father apart. For example, an array always be declared at the beginning of a function but used much later, with multiple statements between the declaration and the application. By weighting attention with distance, the model can focus on distant declaration to capture crucial boundary information for checking safe array usage.

Although *softmax* is commonly applied to normalize attention scores, it produces non-zero values for all scores (i.e., $\text{softmax}(s_i) \neq 0$ for every s in $\mathbf{S}$), which is undesirable when only a few statements are relevant to a vulnerability. While a hard threshold could be applied to truncate small probability values, determining such a hyperparameter is complex. Instead, we adopt a sparse attention mechanism, specially *sparsemax* [30]. This approach preserves key properties of *softmax*, such as non-negativity and sum-to-one while producing sparse distributions. The normalized attention scores are computed as follows:

$$\hat{\mathbf{S}}_p = \text{sparsemax}(\mathbf{S}_p), \tag{5}$$

$$\text{sparsemax}(\hat{s}_{p,q}) = [s_{p,q} - \tau(\mathbf{S}_p)]_+, \tag{6}$$

where $[x]_+ = \max\{0, x\}$, and $\tau(\cdot)$ denotes the threshold function defined in Algorithm 1. Values exceeding the threshold are retained, while others are automatically truncated to zeros.

Algorithm 1: Threshold Evaluation of Sparsemax

Input: $\mathbf{S}$

1 Sort $\mathbf{S}$ as $s_1 \geq \cdots \geq s_n$;

2 Find $\rho = \max\{1 + js_j > \sum_{i=1}^{j} s_i \mid 1 \leq j \leq n\}$;

3 Define $\tau(\mathbf{S}) = \frac{1}{\rho}\left(\sum_{i=1}^{\rho} s_i - 1\right)$;

Output: $\tau(\mathbf{S})$

The node set V_p for sensitive subgraph g_p is defined as:

$$V_p = \{v_p\} \cup \{v_q \mid \hat{s}_{p,q} > 0\}. \tag{7}$$

3.3.2 Sensitive Subgraph Construction

After identifying sensitive nodes, we compute similarity scores to determine edges:

$$e_{p,q} = \text{ReLU}(\mathbf{W}[h_p \parallel h_q]), \tag{8}$$

where $\parallel$ represents the concatenation operation, and $\mathbf{W} \in \mathbb{R}^{2\times d}$ is a learnable weight vector. We then normalize these scores with *sparsemax* to obtain a sparse subgraph:

$$\hat{\mathbf{E}}_p = \text{sparsemax}(\mathbf{E}_p). \tag{9}$$

As the subgraph is sparse, all edges are treated as the same type.

3.3.3 Sensitive Subgraph Learning

We utilize a standard GGNN and global mean pooling to capture subgraph features and update the hidden states of the sensitive nodes:

$$h'_p = h_p + \frac{1}{|V_p|} \sum_{v_q \in V_p} \text{GGNN}(\mathbf{H}_p). \tag{10}$$

The sensitive subgraphs generated with DWSA contain several advantages:

- They are generated automatically, which can provide insight into the decision-making process of the model;
- They overcome the long-range dependency challenge between vulnerable nodes, providing model with comprehensive structural information;
- They are sparse and vulnerability-related, which enrich the feature representations of sensitive nodes.

3.4 The Readout Function and Classifier

3.4.1 Vulnerability Score Calculation

Our model integrates semantic features, operational details, and sensitive subgraphs to predict vulnerabilities.

We compute a semantic score for each node v_p as:

$$w_s(p) = \text{sigmoid}(\text{MLP}(h_p)), \tag{11}$$

where $w_s(p) \in [0, 1]$ represents the semantic score for node v_p, which is computed via a multi-layer perception (MLP), and normalized by *sigmoid* activation function.

To account for operational-specific details, we define an operational weight proportional to the number of operation types exist in node v_p, normalized by the total number of operation types:

$$w_o(p) = \frac{|T_p|}{|T|}, \tag{12}$$

where $w_o(p) \in [0, 1]$ is the operational weight for node v_p, $|T_p|$ denotes the number of the operation type exists in node v_p, and $|T|$ is the total number of operation types defined at Table 1. A larger $w_o(p)$ increases the model's focus on nodes with many operations which are more likely to overlook operational restrictions.

For each sensitive subgraph centered on node v_p, we compute a sensitivity score:

$$w_g(p) = \tanh(\text{MLP}(h_g(p))), \tag{13}$$

where $w_g(p) \in [-1, 1]$ represents the sensitivity score of the subgraph, and $h_g(p)$ denotes the pooled graph features of the sensitive subgraph centered on sensitive node v_p. We assume that a negative sensitive score indicates benign behavior, and a positive score indicates vulnerability. This sensitivity score is a bias to strengthen or weaken

the focus on sensitive nodes. For example, a well-constrained sensitive operation will yield a negative score, reducing its contribution on the final prediction.

The overall score for each node is an aggregation of these components:

$$w = \text{softmax}(w_o \cdot w_s + w_g), \tag{14}$$

with the additional condition that $w_g = 0$ if a node contains no sensitive operation.

3.4.2 Readout and Classify

To generate a fixed-size graph feature, we employ a readout function that aggregates all node representations according to their vulnerability scores:

$$z = \text{ReLU}([\sum_{i=1}^{n} w_i h_i, \max_{i=1}^{n} w_i h_i]). \tag{15}$$

Finally, the graph-level representation z is passed through an MLP to compute the probability distribution over the non-vulnerable and vulnerable classes:

3.5 Learning Objective

We employ a dual-component loss function to maximize the distribution differences between vulnerable and non-vulnerable samples.

For class classification, we utilize the cross-entropy loss:

$$\mathcal{L}_{CE} = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})), \tag{16}$$

where y denotes the ground-true label and $\hat{y}$ is the predicted probability generated by the model.

Additionally, for sample labeled as vulnerable, we require that the vulnerability scores of the nodes exhibit sparsity, while those of benign to be uniform. To achieve this, we incorporate an additional loss component:

$$\mathcal{L}_S = y[\alpha - \sigma]_+ + (1 - y)[\sigma - \beta]_+, \tag{17}$$

where σ represents the standard deviation of the vulnerability scores of the nodes, α and β are hyperparameters set to 0.05 and 0.001 in our experiments. The overall loss function is formulated as:

$$\mathcal{L} = \mathcal{L}_{CE} + \mathcal{L}_S. \tag{18}$$

4 Experiment Setup

4.1 Research Questions

To evaluate our model, we present our empirical results as answers to the following research questions:

- **RQ1: Comparison with Baselines.** What is the performance of SAVED compared with current state-of-the-art vulnerability detection methods?
- **RQ2: Interpretation of Prediction.** Can SAVED provide accurate interpretation for the correctly detected vulnerability?
- **RQ3: Ablation Study.** How effective is the RA-GGNN and DWSA for learning vulnerability patterns?
- **RQ4: Hyperparameter Sensitive Analysis.** How different parameter setups affect the performance of SAVED?
- **RQ5: Case Studies.** What is the performance of SAVED in recent vulnerabilities?
- **RQ6: Effectiveness of Semantic Understanding.** How effective are language models of code in vulnerability detection?

4.2 Dataset

To evaluate our model and ensure a fair comparison with baseline methods, we conduct experiments using three widely recognized datasets: FFmpeg+Qemu [5], Reveal [6] and BigVul [31].

The FFmpeg+Qemu dataset comprises manually labeled samples with a nearly balanced distribution of vulnerable and non-vulnerable instances. The Reveal dataset, collected from two open-source projects (Linux Debian Kernel and Chromium), is highly imbalanced, featuring significantly fewer vulnerable samples than non-vulnerable ones. To mitigate this imbalance, we employ oversampling techniques. The BigVul dataset consists of samples extracted from 348 open-source GitHub projects, including 188,636 C/C++ functions, of which 10,900 are identified as vulnerable across 91 distinct vulnerability types. Notably, BigVul provides patched versions of the vulnerable functions, which are crucial for evaluating the interpretation quality of SAVED. In practice, we retained only those entries that included both vulnerable and patched versions to satisfy the training requirements of the baseline model [9], which can only be trained using slice annotation.

The statistical details of the dataset are presented in Table 2.

Table 2: Statistical details of the datasets

Dataset	Vulnerable	Non-Vulnerable	Total	Vul Rate(%)
FFmpeg+Qemu [5]	10677	12841	23518	45.40
Reveal [6]	1734	18337	20071	8.64
BigVul [31]	10662	10662	21324	50.00

4.3 Implementations

We conduct experiments on a server equipped with a 16-core CPU and an NVIDIA GeForce RTX 4090 GPU. For dataset processing, comment lines are removed from the source code, and improperly truncated or unparsable samples are excluded. We

employ *clang-format* [3] to reformat lines belonging to the same statement that are split across multiple lines for readability when development, and use *Joern* [4] to generate PDGs. The datasets are randomly split into 80% for training, 10% for validating and 10% for testing. We train our model for a maximum of 100 epochs with early stopping implemented if the Matthews Correlation Coefficient (MCC) stops rising for 20 consecutive epochs.

4.4 Metrics

For function-level detection, we evaluate the models using the following metrics:

- **Precision**: $Precision = \frac{TP}{TP+FP}$. This metric quantifies the accuracy of the model's vulnerability predictions.
- **Recall**: $Recall = \frac{TP}{TP+FN}$. This metric measures the ability of the model to identify all vulnerable instances.
- **F_1-Score**: $F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$ The $F1$-Score is the harmonic mean of the precision and recall, providing a balanced measure of both.

For evaluating the interpretation of prediction, we consider only functions correctly predicted as vulnerable and use the following metrics.

- **Accuracy**: Following the implement in [8], an interpretation result is considered as correct if it overlaps with any statement annotated as vulnerable.
- **IoU Score**: We adopt the Intersection over Union(IoU) metric proposed in [32] to assess the localization precision of the vulnerability detection models. It is defined as $IoU = \frac{|U \cap V|}{|U \cup V|}$, where U is the set of truly vulnerable statements and V is the set of the statements predicted as vulnerable.

4.5 Experimental Methodology

4.5.1 RQ1: Comparison with Baselines.

We compare SAVED with three traditional tools and seven learning-based methods:

- **Cppcheck** [1]: Cppcheck is an open-source static analysis tool for C/C++ that detects vulnerabilities and compliance issues by applying customizable rule sets.
- **Flawfinder** [2]: Flawfinder is a security scanner that flags potential vulnerabilities by matching code patterns against a database of known weakness categories.
- **RATS** [3]: RATS (Rough Auditing Tool for Security) is a multi-language static analysis framework that scans source code for common security flaws using rule-based pattern matching.
- **VulDeePecker** [4]: VulDeePecker is a sequence-based model that uses code gadgets, designed based on sensitive APIs, as inputs.

[3] https://clang.llvm.org/docs/ClangFormat.html
[4] https://joern.io

- **SySeVR** [7]: SySeVR is an improved version of VulDeePecker that utilizes code slices derived from four vulnerability syntax characteristics (i.e., Library/API Function Call, Array usage,Pointer Usage, Arithmetic Expression) as inputs.
- **Devign** [5]: Devign employs a GGNN to extract features from the jointed graph of Abstract Syntax Tree (AST), Control-Flow Graphs (CFG), Data Dependency Graph (DDG), and Natural Code Sequence (NCS).
- **Reveal** [6]: Reveal utilizes a GGNN backbone and employs a triplet loss to learn discriminative graph embeddings.
- **IVDetect** [8]: IVDetect constructs fine-grained, context-aware nodes and uses GCN to learn graph representation.
- **AMPLE** [10]: AMPLE simplifies graph representations and designs an edge-aware graph convolutional network (EA-GCN) for graph representation learning.
- **mVulPreter** [9]: mVulPreter is a multi-granularity vulnerability detection model that aggregates the strengths of function-level and slice-level GGNN models.

To ensure a fair comparison, all baseline models except for mVulPreter are trained on all three datasets using their original configurations. Notably, VulDeePecker and SySeVR employ the Bidirectional LSTM as their backbone, which achieves optimal performance in their original studies. The implementations of VulDeePecker, SySeVR and Devign are reproduced using the code provided in [6]. Due to the inherent limitation of mVulPreter, which can only be trained with annotated slices, it is evaluated only on the BigVul dataset.

4.5.2 RQ2: Interpretation of Prediction

In this experiment, we focus on assessing the accuracy of interpretation. We compare our model with two baselines:

- **mVulPreter** [9]: mVulPreter is a vulnerability detection model that can produce interpretations of its detection results through parameter analysis.
- **GNNExplainer** [18]: GNNExplainer is a widely used auxiliary explanation model for interpreting GNN models.

We train a GNNExplainer on a variant of SAVED that employs a readout module that concatenates the mean and maximum pooling of node representations.

To annotate vulnerable statements, we label statements that are deleted or modified in the patched version as vulnerable. In addition, all statements that are control or data dependent on an added validation statement are also labeled as vulnerable. This criterion is based on the observation that many vulnerabilities arise from insufficient input validation[33].

In experiment, we evaluate the accuracy of the top 5 most important statements in the interpreted results of each interpreter as [9].

4.5.3 RQ3: Ablation Study

To evaluate the contributions of the RA-GGNN and the proposed DWSA to vulnerability detection, we retrain ablated versions of SAVED, each omitting one key modules. Specially, we compare:

- **w/o RA-GGNN**: This is a model uses a standard GGNN for local pattern learning, which treats all edge types uniformly.
- **w/o DWSA**: This is a model omits the DWSA and instead employs a readout module that concatenates the mean and max pooling of node representations for the absence of subgraph scores.

Furthermore, to evaluate the effect of DWSA on vulnerability detection in larger codes, we compare the full SAVED and w/o DWSA across varying code scales.

4.5.4 RQ4: Hyperparameter Sensitivity Analysis

When designing the objective of SAVED, we introduce two hyperparameters, α and β, to regularize the distribution of the vulnerability scores. To evaluate their impact, we perform a grid search over the values {1.0, 0.5, 0.1, 0.05, 0.01, 0.005, 0.0001} for both α and β. For each parameter combination, we train and evaluate the model on the FFmpeg+Qemu dataset and report the resulting $F1$-Score metrics.

4.5.5 RQ5: Case Studies

In this experiment, we test SAVED on eleven recent vulnerabilities collected by our experts. We train SAVED on the merged set of the datasets introduced above, then apply it to the eleven recent vulnerabilities.

4.5.6 RQ6: Effectiveness of Semantic Understanding

LMs of code have demonstrated significant success in language-related tasks due to their powerful contextual understanding capabilities. However, we argue that such models may struggle with vulnerability detection, as this task largely relies on structural information and the underling logic of code. To validate this claim, we fine-tune three code-specific pre-trained models and two LLMs for vulnerability detection:

- **CodeBERT** [22]: CodeBERT is a roBERTa [34] based pre-trained model, specially designed for understanding both programming languages and natural language.
- **GraphCodeBERT** [23]: Similar to CodeBERT, GraphCodeBERT is also pre-trained on the roBERTa architecture, but extends its utility by incorporating data flow in the code.
- **PolyCoder** [26]: PolyCoder is a large language model for code generation, trained on 249G of code across 12 programming languages, with particular excellence in the C language.

- **CodeExecutor** [25]: CodeExecutor is a pre-trained model designed to predict execution traces through a code execution pre-training task and curriculum learning.
- **CodeT5+** [24]: CodeT5+ is a large code language model with an encoder-decoder architecture, pre-trained with a diverse set of tasks to support a wide range of code understanding and generation tasks.

For models in the CodeBERT series (i.e., CodeBERT, GraphCodeBERT, and CodeExecutor), we use the first token `[CLS]` as the sequence feature representation. For CodeT5+, we use the encoder of the 220M-parameters version [5] to directly generate sequence embeddings. For PolyCoder, we use the encoder of the 160M-parameter version [6] and perform a mean pooling on the last hidden state embeddings to obtain the sequence embedding. The code embeddings are then fed into a MLP for classification.

Each model is fine-tuned on the BigVul dataset for 10 epochs, with early stopping implemented if the F_1-Score on the validation dataset stop raising for two consecutive epochs.

5 Experiment Results

5.1 RQ1: Comparison with baselines

The performance of SAVED is compared with ten baselines in Table 3.

Table 3: Comparison of function-level vulnerability detection performance with baselines. The best result for each metric is highlighted in **bold** and the second is highlighted in underline

dataset / metric	FFmpeg+Qemu			Reveal			BigVul		
Model	Pr	Re	F1	Pr	Re	F1	Pr	Re	F1
Cppcheck	**82.97%**	7.85%	14.34%	13.18%	5.42%	7.68%	42.68%	23.99%	30.72%
Flawfinder	49.60%	13.37%	21.07%	13.40%	18.92%	15.67%	50.23%	16.75%	25.13%
RATS	49.32%	14.26%	22.13%	13.49%	18.74%	15.69%	50.54%	17.53%	26.03%
VulDeepecker	52.85%	29.40%	37.78%	22.73%	18.52%	20.41%	43.38%	26.58%	32.96%
SySeVR	48.21%	56.25%	51.92%	24.19%	32.61%	27.78%	45.05%	38.12%	41.30%
Devign	49.60%	65.47%	56.44%	17.01%	43.18%	24.41%	-	-	-
Reveal	50.65%	70.11%	58.81%	21.29%	40.00%	27.79%	51.91%	41.36%	46.04%
IVDetect	53.78%	60.36%	56.88%	32.16%	42.95%	36.78%	-	-	-
AMPLE	55.67%	70.89%	62.36%	38.30%	49.09%	43.03%	48.09%	63.45%	54.71%
mVulPreter	-	-	-	-	-	-	51.47%	67.12%	58.34%
SAVED(Ours)	56.27%	**71.38%**	**62.93%**	**40.29%**	**50.61%**	**44.86%**	**57.42%**	**74.37%**	**64.81%**

Traditional static analysis tools, namely Cppcheck, Flawfinder, and RATS, all exhibit high false-negative rates, highlighting the limitations of expert-crafted vulnerability patterns.

[5] https://huggingface.co/Salesforce/codet5p-110m-embedding
[6] https://huggingface.co/NinedayWang/PolyCoder-160M

Sequence-based models, namely VulDeePecker and SySeVR, convert code into specialized code slices based on syntax characteristics associated with vulnerabilities. Among these, SySeVR outperforms VulDeePecker by considering a broader range of characteristic types and recalling more vulnerable samples. However, both models treat slices as text sequences, overlooking the underling structural dependencies between statements, which leads to suboptimal performance.

Graph-based models, including Devign, Reveal, IVDetect and AMPLE, generally outperform sequence-based models, demonstrating the effectiveness of the structural insights captured by GNNs.

It is noteworthy that these models experience a significant performance reduction on the BigVul dataset, particularly Devign and IVDetect. We fail to train Devign and IVDetect on the BigVul dataset, which tend to predict all samples as vulnerable even after adjusting the weight of the loss function. We hypothesize that this issue arises because the BigVul dataset contains both vulnerable and patched versions of vulnerabilities which are similar, requiring models to understand the underling structural and logical nuances of the changes. Devign and IVDetect rely solely on the GNN as a graph encoder, resulting in an inability to capture these subtle changes, ultimately reducing their performance to that of a semantic model. In contrast, VulDeePecker and SySeVR use code slices as input, which allows them to identify a small number of vulnerabilities instead. Reveal employs a triplet loss function to maximum the differences between samples from different classes, while AMPLE simplifies the graph representations to minimize the distance between nodes, all leading to comparable performance.

Additionally, mVulPreter aggregates the results of both slice-level and function-level models, achieving the best performance among the baselines.

In comparison with these baseline models, SAVED achieves superior performance across all metrics, benefiting from local pattern learning with GGNN and sensitive subgraph learning with DWSA.

5.2 RQ2: Interpretation of Prediction

This experiment evaluates the accuracy and precision of the interpretations produced by SAVED. As shown in Table 4, SAVED outperforms both mVulPreter and GNNExplainer. mVulPreter calculates vulnerability scores by aggregating the results from

Table 4: Comparison of the interpretation of prediction.

Model	Interpretation	Accuracy	IoU
mVulPreter	parameter analysis	65.00%	-
GNNExplainer	auxiliary model	49.75%	14.31%
SAVED(Ours)	end-to-end	**71.91%**	**24.43%**

a slice-level model and a function-level model, demonstrating the superiority of the

multi-granularity in vulnerability detection. However, mVulPreter only focuses on the output layer rather than investigating the detection process of the models. GNNExplainer, as an auxiliary explanation model, is trained to interpret the prediction of the primary detection model. However, such auxiliary explanation model focus on explaining individual instances instead of understanding the model's detection process. In contrast, SAVED generates interpretation by explicitly aggregating the semantic information, operational details and sensitive subgraphs, which are directly involved in the decision-making process.

5.3 RQ3: Ablation Study

In the ablation study, we quantify the contributions of the RA-GGNN and DWSA components in SAVED. As shown in Table 5, incorporating RA-GGNN and DWSA substantially improves performance on both the FFmpeg+Qemu and Reveal datasets.

Additionally, omitting DWSA reduces SAVED to a basic GNN model that only captures local patterns, causing the model to fail to generate predictions on the BigVul dataset, similar to Devign and IVDetect.

Table 5: Results of ablation study.

metric \ dataset	FFmpeg+Qemu			Reveal		
Model	Pr	Re	F1	Pr	Re	F1
w/o RA-GGNN	49.61	70.72	58.31	34.78	46.78	39.90
w/o DWSA	53.26	62.69	57.60	35.06	47.37	40.30
SAVED	**56.27**	**71.38**	**62.93**	**40.29**	**50.61**	**44.86**

We further assess DWSA's effectiveness on larger-scale code. Figure 4 shows that the performance of the base model severely decreases as code length increases. By contrast, SAVED maintains robust detection performance across varying code lengths.

These experimental results indicate that modeling code as a heterogeneous graph enhances the SAVED's knowledge towards vulnerability and the proposed DWSA can effectively exclude the noise introduced by irrelevant statements, therefore improving SAVED's capability to identify vulnerabilities across varying length codes.

5.4 RQ4: Hyperparameters Sensitivity Analysis

Results in Figure 5 reveals three key observations:

1. The model achieves the best $F1$-Score when $\alpha \geq 0.05$ and $\beta \leq 0.01$. In our loss function, α amplifies the disparity among vulnerability scores of vulnerable samples, while β attenuates the disparity among scores of benign samples. These experimental findings confirm that our design enhances the model's capability to discriminate vulnerability patterns.
2. With β fixed, the $F1$-Score increases as α decreases, peaking at approximately $\alpha = 0.01$. This trend suggests that an excessively large α over-constrains the

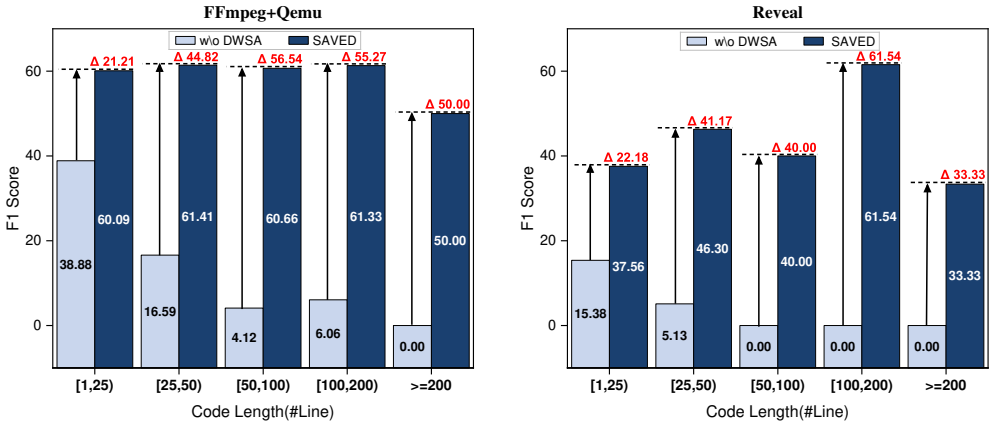


Fig. 4: $F1$-Score comparison with different code length in the FFmpeg+Qemu and Reveal dataset.

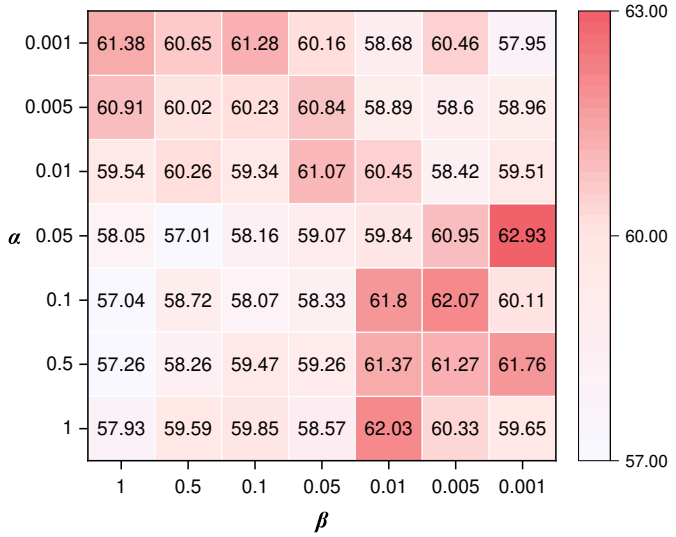


Fig. 5: $F1$-Score under different parameter settings on FFmpeg+Qemu dataset.

scores of vulnerable samples, whereas an excessively small α fails to enforce sufficient separation, resulting in performance degradation.

3. For $\alpha \geq 0.05$, performance improves as β decreases, consistent with our design. However, beyond a certain α threshold, a larger β yields unexpectedly better results. We hypothesize that when α inadequately constrains the outputs, the score differences diminish and SAVED attends to irrelevant node features. In such cases, increasing the constraint on benign samples via β becomes overly restrictive and further impairs genuine pattern learning.

5.5 RQ5: Case Studies

The details of the eleven collected vulnerabilities and the detection results of SAVED are presented in Table 6. Of the four projects, Asterisk represents a cross-domain

Table 6: The details of the eleven collected vulnerabilities and the detection results

Program	CVE ID	Vulnerable function	Prediction
Asterisk	CVE-2021-37706	decode_errcode_attr	predicted
	CVE-2023-37457	update_header	missed
curl	CVE-2019-5481	read_data	missed
	CVE-2021-22898	suboption	predicted
FFmpeg	CVE-2022-2566	build_open_gop_key_points	predicted
	CVE-2023-47342	parse_fmtp	predicted
	CVE-2024-28661	parse_speex_extradata	predicted
	CVE-2024-31582	draw_block_rectangle	predicted
	CVE-2024-36617	read_pakt_chunk	predicted
qemu	CVE-2023-3180	crypto_sym_op_helper	predicted
	CVE-2023-6683	virtio_net_set_mrg_rx_bufs	predicted
Recall			81.82%

scenario, as SAVED has not been trained on any historical vulnerabilities of Asterisk. The remaining three projects (i.e., curl, FFmpeg, and qemu) are in-domain scenarios. SAVED correctly predicts nine of the eleven vulnerabilities. Specially, SAVED achieves perfect detection on FFmpeg and qemu, demonstrating its ability in in-domain vulnerability detection. However, in cross-domain capability, SAVED still requires further improvement.

Additionally, SAVED identifies potential incorrect CVE information of curl (i.e., CVE-2019-5481). The CVE description asserts that the patch avoids potential double-free errors arising from failed `realloc` calls. However, the referenced commit in Figure 6 solely enhances the compatibility between the function and the newly introduced `Curl_saferealloc` which is defined as `void* Curl_saferealloc(void *ptr, size_t size)` [7]. The actual vulnerability fix occurs earlier when `realloc` is replaced by `Curl_saferealloc` [8]. Therefore, the compatibility enhancement cannot be recognized as a security patch. This issue is an example of the label-quality challenge, which will be discussed in Section 7.

5.6 RQ6: Effectiveness of Semantic Understanding

This experiment is conducted to validate our assumption that semantic understanding alone is insufficient for vulnerability detection. We conduct the experiment only on the BigVul dataset, as it contains both vulnerable and patched versions of vulnerabilities. A model capable of distinguishing vulnerabilities must identify subtle structural and logical differences between the two versions. The performance of five language

[7] https://github.com/curl/curl/commit/9069838b30fb3b48af0123e3
[8] https://github.com/curl/curl/commit/0649433da53c7165f839e2

```
static CURLcoderead_data(struct connectdata *conn,
                         curl_socket_t fd,
                         struct krb5buffer *buf)
{
    int len;
    void *tmp = NULL;
    CURLcode result;
    result = socket_read(fd, &len, sizeof(len));
    if(result)
        return result;
    if(len) {
    /* only realloc if there was a length */
        len = ntohl(len);
-   tmp = Curl_saferealloc(buf->data, len);
+   buf->data = Curl_saferealloc(buf->data, len);
    }
-   if(tmp == NULL)
+   if(!len || !buf->data)
        return CURLE_OUT_OF_MEMORY;
-   buf->data = tmp;
    result = socket_read(fd, buf->data, len);
    if(result)
        return result;
    ...
    return CURLE_OK;
}
```

Fig. 6: Patch for CVE-2019-5481, with **added** lines and **deleted** lines highlighted. In the original version, the variables `len` and `tmp` are both well checked. Therefore, the modification cannot be recognized as a security patch.

models on the BigVul dataset is presented in Table 7. Among these five models, CodeBERT, PolyCoder, and CodeT5+ perform pooly in distinguishing vulnerability samples. Specially, CodeBERT almost predicts all functions as vulnerable, while PolyCoder and CodeT5+ exhibit a random prediction. This result demonstrates that semantic understanding alone is not enough for vulnerability detection. In contrast, GraphCodeBERT and CodeExecutor perform slightly better, as GraphCodeBERT incorporates additional data dependency information, while CodeExecutor excels in identifying execution traces. These models are not merely pre-trained for semantic understanding but also incorporate structural understanding, enable them to perform better in vulnerability detection.

6 Related Works

In this section, we review the related work about vulnerability detection.

Table 7: Effectiveness of language models in vulnerability detection.

Model	#Param.	Precision	Recall	F1
CodeBERT	125M	50.89%	95.45%	66.39%
GraphCodeBERT	125M	52.76%	70.04%	60.18%
PolyCoder	160M	49.66%	48.05%	48.84%
CodeExecutor	126M	50.49%	81.17%	62.26%
CodeT5+	110M	50.48%	56.31%	53.24%

6.1 Learning-based Vulnerability Detection

VulDeePecker[4] and SySeVR[7] construct code slices based on vulnerability syntax characteristics and treat these slices as flat token sequences. However, sequence-based models suffer from information loss when processing long text. In contrast, GNN models [5, 6, 8–10, 17] convert the source code into graph representations and train Graph Neural Network, which leverage both semantic and structural information to automatically learn vulnerability-related features from the graph embedding. However, these models cannot overcome the long-range dependency challenge between the root cause of a vulnerability and its manifestation point.

6.2 Fine-grained Vulnerability Detection

Vulnerabilities always arise from a small subset of statements of the whole codes. Some studies are focusing on optimizing code graph representations to mitigate the noise from irrelevant statements. AMPLE[10] designs type-based and variable-based graph simplification rules to reduce the distances between nodes. The simplification rules are aimed at abstract syntax tree node, still unable to solve the coarse-grained representation with numerous irrelevant statements. GDSV [35] uses the sliding window composition method to create a novel token-based graph and learns features in a finer-grained level. Slice-level models [4, 7, 9, 32] are finer-grained, but code slices may still contain numerous statements and multiple sensitive operations. ANGLE [36] proposes a self-adaptive pooling operation to simplifies the graph representation when learning.

6.3 Model Interpretation

Providing interpretable predictions is another important indicator for measuring the effectiveness of vulnerability detection models. IVDetect uses GNNExplainer[18] to fine a subgraph that minimizes the difference in prediction scores from the entire graph. CFExplainer[37] investigate counterfactual reasoning and provide counterfactual explanation for the vulnerability detection task. In contrast, methods based on parameter analysis [9, 19, 32, 38] explain predictions by calculating the contribution of each statement according to the models' parameters. However, these methods provide post-hoc explanations that are disjointed from the models' prediction, which may lead to misrepresentations.

7 Discussion

In this section, we discuss the limitation of our model.

Single Function Paradigm. Each dataset used in our experiment follows a single function paradigm, where each sample contains only one function. However, in reality, vulnerabilities often span across multiple functions. For example, memory is typically allocated and freed in different functions. This limitation may lead to an unfair evaluation standard, potentially causing model to exhibit a higher false positive rate or false negative rate.

Graph Parsing. Graph-based vulnerability detection models typically rely on *Joern* for graph parsing. However, we find that *Joern* may fail to parse code when marco definitions and structure definitions are missing. This limitation leads to varying degrees of data loss across different methods. Additionally, we find that the PDG generated by *Joern* lacks nodes related to variable declarations, which are crucial for capturing information about variable types. In future work, we will explore other code parsing tools, such as *LLVM* [39] and *Frama-C* [40] to achieve more robust graph generation.

Label Quality. The datasets used in our experiments suffer from poor label quality [41]. In previous studies, samples are labeled as vulnerable if they were changed by a commit identified as fixing a vulnerability. However, vulnerability fixes may affect multiple functions, some of which may be benign but are changed to facilitate the correction of the vulnerability. This noice may negatively impact model performance. GCL4SVD [42] proposes graph confident learning and VulChecker [17] proposes data augmentation technique that may help improve the robustness of models, which will be explored in our future work.

Interpretation. While current deep learning-based models perform much better than traditional vulnerability detection tools, they still exhibit a high false positive and false negative rate when tested in real-world scenarios. We argue that this is normal, as current models lack sufficient contextual information and make predictions based only on a single function. We also contend that it is acceptable if the model can provide adequate explanations for predictions that are transparent to a security analyst. Vulnerabilities are often highly context-dependent, and whether a sensitive operation is vulnerable must be determined based on the specific scenario in which it occurs, requiring security analysts to make the final determination. Therefore, developing models with robust interpretability is as important as developing reliable detection models.

8 Conclusion

In this paper, we propose an end-to-end GNN vulnerability detection model SAVED that prioritizes sensitive operations in code and simultaneously provides both predictions and corresponding explanations. We demonstrate the effectiveness of SAVED by comparing it with current state-of-the-art methods. Our work demonstrates that the tasks of prediction and explanation can be concurrently enhanced during the training of vulnerability detection models.

9 Data Availability

The source code will be available when the paper is accepted.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant 62276060.

Author contributions

Kaiyang Cai, Jinghao Hu, Jingyao Li and Zhanshan Li conceived the research and designed the experiments. Kaiyang Cai constructed the initial framework of the paper, carried out the numerical experiments, and drafted the manuscript. Jinghao Hu and Jingyao Li refined the design and experiments, and comprehensively revised the manuscript. Jingyao Li and Zhanshan Li supervised the research and performed the final manuscript editing. All authors reviewed and confirmed the methods and conclusions.

Conflict of interest

The authors declare no competing interests.

References

[1] Cppcheck: Cppcheck. https://cppcheck.sourceforge.io/

[2] Flawfinder: FlawFinder. https://dwheeler.com/flawfinder/

[3] RATAS: Rough Audit Tool for Security. https://code.google.com/archive/p/rough-auditing-tool-for-security/

[4] Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: Vuldeepecker: A deep learning-based system for vulnerability detection. arXiv preprint arXiv:1801.01681 (2018)

[5] Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Advances in neural information processing systems **32** (2019)

[6] Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: Are we there yet? IEEE Transactions on Software Engineering **48**(9), 3280–3296 (2021)

[7] Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: Sysevr: A framework for using deep learning to detect software vulnerabilities. IEEE Transactions on Dependable and Secure Computing **19**(4), 2244–2258 (2021)

[8] Li, Y., Wang, S., Nguyen, T.N.: Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 292–303 (2021)

[9] Zou, D., Hu, Y., Li, W., Wu, Y., Zhao, H., Jin, H.: mvulpreter: A multi-granularity vulnerability detection system with interpretations. IEEE Transactions on Dependable and Secure Computing (2022)

[10] Wen, X.-C., Chen, Y., Gao, C., Zhang, H., Zhang, J.M., Liao, Q.: Vulnerability detection with graph simplification and enhanced graph representation learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2275–2286 (2023). IEEE

[11] Yamaguchi, F., Golde, N., Arp, D., Rieck, K.: Modeling and discovering vulnerabilities with code property graphs. In: 2014 IEEE Symposium on Security and Privacy, pp. 590–604 (2014). IEEE

[12] Ferrante, J., Ottenstein, K.J., Warren, J.D.: The program dependence graph and its use in optimization. ACM Transactions on Programming Languages and Systems (TOPLAS) **9**(3), 319–349 (1987)

[13] Li, Y., Tarlow, D., Brockschmidt, M., Zemel, R.: Gated graph sequence neural networks. arXiv preprint arXiv:1511.05493 (2015)

[14] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907 (2016)

[15] Mikolov, T.: Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781 **3781** (2013)

[16] Grover, A., Leskovec, J.: node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864 (2016)

[17] Mirsky, Y., Macon, G., Brown, M., Yagemann, C., Pruett, M., Downing, E., Mertoguno, S., Lee, W.: {VulChecker}: Graph-based vulnerability localization in source code. In: 32nd USENIX Security Symposium (USENIX Security 23), pp. 6557–6574 (2023)

[18] Ying, Z., Bourgeois, D., You, J., Zitnik, M., Leskovec, J.: Gnnexplainer: Generating explanations for graph neural networks. Advances in neural information processing systems **32** (2019)

[19] Fu, M., Tantithamthavorn, C.: Linevul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories, pp. 608–620 (2022)

[20] Huang, Z., Li, K., Wang, S., Jia, Z., Zhu, W., Mehrotra, S.: Ses: Bridging the gap between explainability and prediction of graph neural networks. In: 2024 IEEE 40th International Conference on Data Engineering (ICDE), pp. 2945–2958 (2024). IEEE

[21] Hu, Y., Wang, S., Li, W., Peng, J., Wu, Y., Zou, D., Jin, H.: Interpreters for gnn-based vulnerability detection: Are we there yet? In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1407–1419 (2023)

[22] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al.: Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155 (2020)

[23] Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al.: Graphcodebert: Pre-training code representations with data flow. arXiv preprint arXiv:2009.08366 (2020)

[24] Wang, Y., Le, H., Gotmare, A.D., Bui, N.D., Li, J., Hoi, S.C.: Codet5+: Open code large language models for code understanding and generation. arXiv preprint arXiv:2305.07922 (2023)

[25] Liu, C., Lu, S., Chen, W., Jiang, D., Svyatkovskiy, A., Fu, S., Sundaresan, N., Duan, N.: Code execution with pre-trained language models. arXiv preprint arXiv:2305.05383 (2023)

[26] Xu, F.F., Alon, U., Neubig, G., Hellendoorn, V.J.: A systematic evaluation of large language models of code. In: Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, pp. 1–10 (2022)

[27] Rahman, M.M., Ceka, I., Mao, C., Chakraborty, S., Ray, B., Le, W.: Towards causal deep learning for vulnerability detection. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–11 (2024)

[28] Yuan, B., Lu, Y., Fang, Y., Wu, Y., Zou, D., Li, Z., Li, Z., Jin, H.: Enhancing deep learning-based vulnerability detection by building behavior graph model. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2262–2274 (2023). IEEE

[29] Chung, J., Gulcehre, C., Cho, K., Bengio, Y.: Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint arXiv:1412.3555 (2014)

[30] Martins, A., Astudillo, R.: From softmax to sparsemax: A sparse model of attention and multi-label classification. In: International Conference on Machine Learning, pp. 1614–1623 (2016). PMLR

[31] Fan, J., Li, Y., Wang, S., Nguyen, T.N.: A c/c++ code vulnerability dataset with code changes and cve summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 508–512 (2020)

[32] Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H.: Vuldeelocator: a deep learning-based fine-grained vulnerability detector. IEEE Transactions on Dependable and Secure Computing **19**(4), 2821–2837 (2021)

[33] Yamaguchi, F., Wressnegger, C., Gascon, H., Rieck, K.: Chucky: Exposing missing checks in source code for vulnerability discovery. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security, pp. 499–510 (2013)

[34] Liu, Y.: Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692 (2019)

[35] Liu, H., Jiang, S., Qi, X., Qu, Y., Li, H., Li, T., Guo, C., Guo, S.: Detect software vulnerabilities with weight biases via graph neural networks. Expert Systems with Applications **238**, 121764 (2024)

[36] Peng, X., Wang, S., Qin, Y., Lin, B., Chen, L., Mao, X.: Keep it simple: Towards accurate vulnerability detection for large code graphs. arXiv preprint arXiv:2412.10164 (2024)

[37] Chu, Z., Wan, Y., Li, Q., Wu, Y., Zhang, H., Sui, Y., Xu, G., Jin, H.: Graph neural networks for vulnerability detection: A counterfactual explanation. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 389–401 (2024)

[38] He, H., Wang, S., Wang, Y., Liu, K., Yu, L.: Vultr: Software vulnerability detection model based on multi-layer key feature enhancement. Computers & Security **148**, 104139 (2025)

[39] LLVM: The LLVM compiler infrastructure. https://llvm.org/

[40] Frama-C: Frama-c. https://frama-c.com/

[41] Croft, R., Babar, M.A., Kholoosi, M.M.: Data quality for software vulnerability datasets. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 121–133 (2023). IEEE

[42] Wang, Q., Li, Z., Liang, H., Pan, X., Li, H., Li, T., Li, X., Li, C., Guo, S.: Graph confident learning for software vulnerability detection. Engineering Applications

of Artificial Intelligence **133**, 108296 (2024)